

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**  
**MODUL 10**  
**JAVA COLLECTIONS**



**Disusun Oleh:**  
**Iman Dwi Satrio 105224029**

**PROGRAM STUDI ILMU KOMPUTER**  
**FAKULTAS SAINS DAN KOMPUTER**  
**UNIVERSITAS PERTAMINA**

**2026**

## I. Pendahuluan

Praktikum Pemrograman Berorientasi Objek (PBO) pada modul ini berfokus pada pemahaman dan implementasi Java Collections Framework (JCF) sebagai solusi penyimpanan data objek yang dinamis dan efisien. Berbeda dengan struktur array konvensional yang bersifat statis dengan kapasitas memori yang kaku sejak awal deklarasi, Java Collections menawarkan fleksibilitas tinggi dalam memanipulasi susunan data kompleks di memori lokal secara real-time. Implementasi studi kasus pada praktikum ini berpusat pada perancangan program simulasi Sistem Manajemen Gudang murni berbasis pemrograman berorientasi objek yang memanfaatkan tiga pilar utama arsitektur Collections, yaitu List, Set, dan Map.

Ketiga pilar tersebut dikombinasikan secara spesifik guna memenuhi kebutuhan fungsional sistem database gudang internal yang andal. Antarmuka Map, yang diimplementasikan melalui kelas HashMap, digunakan sebagai database penampung data utama karena karakteristik pencarian berbasis pasangan kunci-nilai (key-value pair) yang sangat cepat melalui proses hashing. Selanjutnya, komponen Set melalui instansiasi HashSet difungsikan untuk merekapitulasi kategori produk secara unik guna memastikan tidak adanya duplikasi data kategori di dalam sistem. Terakhir, komponen List dengan pemanfaatan kelas ArrayList digunakan untuk mengkompilasi seluruh riwayat aktivitas transaksi barang masuk dan keluar secara terurut kronologis berdasarkan urutan waktu kejadian.

## II. Variabel

| No | Nama Variabel    | Tipe data | Fungsi                                                                     |
|----|------------------|-----------|----------------------------------------------------------------------------|
| 1  | idBarang         | String    | Menyimpan identitas unik (ID) dari setiap barang di gudang                 |
| 2  | namaBarang       | String    | Menyimpan informasi nama dari barang                                       |
| 3  | kategori         | String    | Menyimpan nama kelompok atau kategori pengelompokan barang                 |
| 4  | stok             | int       | Menyimpan jumlah kuantitas sisa stok barang yang tersedia                  |
| 5  | databaseBarang   | Map       | Sebagai In-Memory Database dengan ID barang sebagai kunci uniknya          |
| 6  | kategoriUnik     | Set       | Himpunan dinamis untuk merekapitulasi daftar kategori tanpa duplikasi      |
| 7  | riwayatAktivitas | List      | Daftar terurut untuk menampung seluruh catatan transaksi secara kronologis |

|    |            |              |                                                                               |
|----|------------|--------------|-------------------------------------------------------------------------------|
| 8  | id         | String       | Parameter input pada method untuk pencarian barang berdasarkan ID             |
| 9  | nama       | String       | Parameter input untuk menampung nama saat mendaftarkan barang baru.           |
| 10 | jumlah     | int          | Parameter input penampung kuantitas barang yang ingin ditambah atau dikurangi |
| 11 | barangBaru | Barang       | Objek penampung sementara saat instansiasi barang baru dilakukan              |
| 12 | barang     | Barang       | Objek penampung sementara saat instansiasi barang baru dilakukan              |
| 13 | gudang     | SistemGudang | Objek utama sistem manajemen gudang yang diinstansiasi di kelas Main          |

### III. Constructor dan Method

| No | Nama Metode     | Jenis Metode | Fungsi                                                                                            |
|----|-----------------|--------------|---------------------------------------------------------------------------------------------------|
| 1  | Barang()        | Constructor  | Menginisialisasi objek model barang baru dengan data ID, nama, kategori, dan stok awal            |
| 2  | getIdBarang()   | Functional   | Mengambil nilai atribut privat idBarang                                                           |
| 3  | getNamaBarang() | Functional   | Mengambil nilai atribut privat namaBarang                                                         |
| 4  | getKategori()   | Functional   | Mengambil nilai atribut privat kategori                                                           |
| 5  | getStok()       | Functional   | Mengambil nilai atribut privat stok                                                               |
| 6  | setStok()       | Procedural   | Mengubah nilai kuantitas pada atribut privat stok                                                 |
| 7  | SistemGudang()  | Constructor  | Menginisialisasi objek sistem gudang sekaligus membuat instansiasi kosong bagi Map, Set, dan List |

|    |                        |            |                                                                                              |
|----|------------------------|------------|----------------------------------------------------------------------------------------------|
| 8  | tambahBarang<br>Baru() | Procedural | Mendaftarkan objek barang ke Map database dan memasukkan kategorinya ke dalam komponen Set   |
| 9  | tambahStok()           | Procedural | Menambah jumlah kuantitas stok barang tertentu di dalam Map berdasarkan pencarian ID         |
| 10 | kurangiStok()          | Procedural | Mengurangi kuantitas stok jika ID valid dan jumlah sisa stok mencukupi                       |
| 11 | cetakLaporan(<br>)     | Procedural | Menampilkan rangkuman kategori unik, sisa stok tabel barang, dan kronologi riwayat transaksi |
| 12 | main()                 | Procedural | Program utama guna menjalankan seluruh rangkaian simulasi sistem gudang                      |

#### IV. Dokumentasi dan Pembahasan Code

Bagian ini akan membahas logika pemrograman yang diimplementasikan.

```

1  class Barang {
2      private String idBarang;
3      private String namaBarang;
4      private String kategori;
5      private int stok;
6
7      public Barang(String idBarang, String namaBarang, String kategori, int stok) {
8          this.idBarang = idBarang;
9          this.namaBarang = namaBarang;
10         this.kategori = kategori;
11         this.stok = stok;
12     }
13
14     public String getIdBarang() { return idBarang; }
15     public String getNamaBarang() { return namaBarang; }
16     public String getKategori() { return kategori; }
17     public int getStok() { return stok; }
18     public void setStok(int stok) { this.stok = stok; }
19 }

```

**Penjelasan:** Kode di atas adalah deklarasi kelas Barang beserta atribut-atribut pendukungnya. Kelas ini dirancang sebagai kelas model (Data Object) untuk merepresentasikan entitas barang fisik yang disimpan di dalam gudang. Kelas ini menerapkan prinsip dasar OOP murni berupa Enkapsulasi Data (Data Encapsulation) guna mengamankan state objek dari modifikasi luar secara ilegal.

- Baris 2–5: Seluruh atribut internal barang (idBarang, namaBarang, kategori, dan stok) dideklarasikan menggunakan akses modifier private. Hal ini bertujuan untuk menyembunyikan data internal objek (data hiding) agar tidak dapat dimodifikasi atau dibaca secara acak oleh kelas eksternal tanpa melalui prosedur kendali resmi.
- Baris 7–12: Konstruktor public Barang() berfungsi sebagai media inisialisasi parameter saat sebuah objek produk baru diinstansiasi ke dalam memori. Penggunaan kata kunci this (seperti this.idBarang = idBarang) diperlukan untuk membedakan antara atribut kelas global dengan variabel parameter lokal yang memiliki nama identik.
- Baris 14–18: Sebagai enkapsulasi, disediakan metode akses kontrol bertipe functional/procedural. Metode getIdBarang(), getNamaBarang(), getKategori(), dan getStok() bertindak sebagai metode read-only (Getter) agar kelas lain dapat mengambil data nilai tanpa risiko mengubahnya. Sebaliknya, metode setStok() bertindak sebagai Setter untuk mengubah kuantitas stok barang saat terjadi aktivitas mutasi barang di dalam gudang.

```

1 import java.util.*;
2
3 class SistemGudang {
4     private Map<String, Barang> databaseBarang;
5     private Set<String> kategoriUnik;
6     private List<String> riwayatAktivitas;
7
8     public SistemGudang() {
9         this.databaseBarang = new HashMap<>();
10        this.kategoriUnik = new HashSet<>();
11        this.riwayatAktivitas = new ArrayList<>();
12    }
13
14    public void tambahBarangBaru(String id, String nama, String kategori, int stok) {
15        if (!databaseBarang.containsKey(id)) {
16            Barang barangBaru = new Barang(id, nama, kategori, stok);
17            databaseBarang.put(id, barangBaru);
18            kategoriUnik.add(kategori);
19            riwayatAktivitas.add("Barang Baru Ditambah: " + id + " - " + nama + " (" + kategori + ") sebanyak " + stok + " unit");
20        } else {
21            riwayatAktivitas.add("Gagal Daftar: ID " + id + " sudah terpakai.");
22        }
23    }
24
25    public void tambahStok(String id, int jumlah) {
26        Barang barang = databaseBarang.get(id);
27        if (barang != null) {
28            barang.setStok(barang.getStok() + jumlah);
29            riwayatAktivitas.add("Barang Masuk: " + id + " ditambah " + jumlah + " unit");
30        } else {
31            riwayatAktivitas.add("Gagal Tambah Stok: ID " + id + " tidak ditemukan.");
32        }
33    }
34
35    public void kurangiStok(String id, int jumlah) {
36        Barang barang = databaseBarang.get(id);
37        if (barang == null) {
38            riwayatAktivitas.add("Gagal Kurang Stok: ID " + id + " tidak ditemukan.");
39            System.out.println("Transaksi Gagal: ID " + id + " tidak ditemukan.");
40            return;
41        }
42
43        if (barang.getStok() >= jumlah) {
44            barang.setStok(barang.getStok() - jumlah);
45            riwayatAktivitas.add("Barang Keluar: " + id + " dikurangi " + jumlah + " unit");
46        } else {
47            riwayatAktivitas.add("Gagal Kurang Stok: " + id + " stok tidak mencukupi (Sisa: " + barang.getStok() + ", Diminta: " + jumlah + ")");
48            System.out.println("Transaksi Gagal: Stok " + id + " tidak mencukupi.");
49        }
50    }
51
52    public void cetakLaporan() {
53        System.out.println("DAFTAR KATEGORI UNIK");
54        for (String kat : kategoriUnik) {
55            System.out.println("- " + kat);
56        }
57
58        System.out.println("SISA STOK BARANG");
59        for (Barang b : databaseBarang.values()) {
60            System.out.println(b.getIdBarang() + " - " + b.getNamaBarang() + " (" + b.getKategori() + ") : " + b.getStok() + " unit");
61        }
62
63        System.out.println("RIWAYAT AKTIVITAS TRANSAKSI");
64        for (String riwayat : riwayatAktivitas) {
65            System.out.println("- " + riwayat);
66        }
67    }
68 }

```

**Penjelasan:** Kode di atas adalah deklarasi kelas SistemGudang beserta atribut-atribut pendukungnya. Kelas SistemGudang tersebut merupakan kelas sebagai controller yang mengatur seluruh logika bisnis mutasi inventaris. Kelas ini mengintegrasikan tiga pilar utama komponen Java Collections Framework untuk memastikan pemrosesan data berjalan dinamis, aman, dan efisien.

- Baris 4–11: Atribut kelas dideklarasikan menggunakan tipe interface dasar (Map, Set, List) sesuai instruksi soal, lalu diinstansiasi di dalam konstruktor menggunakan kelas konkretnya masing-masing. HashMap dipilih agar pencarian objek barang berbasis ID memiliki kompleksitas waktu konstan dengan pasangan (key, value). HashSet digunakan karena memiliki kemampuan bawaan untuk memfilter dan menolak elemen teks duplikat secara otomatis. ArrayList digunakan untuk menampung rekaman kronologis transaksi tanpa batas ukuran.
- Metode tambahBarangBaru (Baris 14–23): Metode ini memvalidasi keunikan ID terlebih dahulu menggunakan fungsi .containsKey(id). Jika ID belum terdaftar, instansiasi objek Barang dilakukan. Objek dimasukkan ke dalam

Map database via `.put()`, teks kategori diekstrak ke dalam Set via `.add()`, dan mutasi sukses dicatat ke dalam log riwayat. Jika ID duplikat ditemukan, operasi diabaikan dan pesan kegagalan dimasukkan ke dalam List riwayat.

- Metode `tambahStok` (Baris 25–33): Pencarian data dilakukan secara instan melalui kunci ID menggunakan perintah `.get(id)`. Jika referensi objek barang ditemukan (tidak bernilai null), sistem memanggil metode mutasi `.setStok()` milik objek terkait dengan menjumlahkan nilai stok lama dengan nilai parameter jumlah masukan baru.
- Metode `kurangiStok` (Baris 35–50): Metode ini menerapkan pengkondisian berlapis. Validasi pertama memastikan keberadaan data ID di dalam Map gudang. Validasi kedua menguji kelayakan kuantitas data fisik dengan membandingkan sisa stok internal via `.getStok()` terhadap kuantitas penarikan barang. Jika kondisi bernilai salah (false), transaksi digagalkan dan pesan peringatan dicetak langsung ke layar konsol melalui perintah `System.out.println()`.
- Metode `cetakLaporan` (Baris 52–67): Fungsi ini melakukan ekstraksi dan cetak data secara menyeluruh dengan teknik iterasi For-Each Loop. Untuk menyajikan isi database barang, digunakan fungsi `.values()` guna mengambil seluruh kumpulan koleksi objek model `Barang` yang tersimpan di dalam struktur `HashMap` tanpa mepedulikan data kuncinya.

```
1  public class Main {
2      public static void main(String[] args) {
3          SistemGudang gudang = new SistemGudang();
4
5          gudang.tambahBarangBaru("B01", "HP", "Elektronik", 10);
6          gudang.tambahBarangBaru("B02", "Meja", "Furnitur", 5);
7          gudang.tambahBarangBaru("B03", "Kipas", "Elektronik", 25);
8
9          gudang.tambahStok("B01", 5);
10
11         gudang.kurangiStok("B03", 10);
12
13         gudang.kurangiStok("B02", 12);
14
15         gudang.cetakLaporan();
16     }
17 }
18
```

**Penjelasan:** Kelas `Main` berfungsi sebagai titik eksekusi program. Di dalam metode `main`, dilakukan instansiasi objek dari kelas pengendali bisnis logistik gudang serta simulasi serangkaian skenario transaksi mutasi barang secara berurutan.

- Baris 3: Baris kode `SistemGudang gudang = new SistemGudang();` melakukan instansiasi objek utama gudang. Proses eksekusi ini memicu metode konstruktor `SistemGudang()` untuk mengalokasikan ruang kosong bagi struktur data `HashMap`, `HashSet`, dan `ArrayList` ke dalam memori komputer.
- Baris 5–7: Program memanggil metode `tambahBarangBaru()` sebanyak tiga kali untuk mendaftarkan tiga item berbeda, yaitu "HP" (Elektronik), "Meja" (Furnitur), dan "Kipas" (Elektronik). Pada tahap ini, mekanisme internal `HashSet` menyaring kategori masukan sehingga hanya menyimpan dua nama kategori unik, yaitu "Elektronik" dan "Furnitur", meskipun kategori "Elektronik" dimasukkan sebanyak dua kali.
- Baris 9–11: Metode `tambahStok("B01", 5)` dipanggil untuk menambahkan kuantitas barang dengan ID B01 sebanyak 5 unit, sehingga total stok internal objek HP melonjak dari 10 menjadi 15 unit. Metode `kurangiStok("B03", 10)` dieksekusi untuk memotong ketersediaan barang dengan ID B03 (Kipas) sebesar 10 unit. Karena sisa stok awal (25 unit) lebih besar dari jumlah penarikan, transaksi dinyatakan sukses dan sisa stok diperbarui menjadi 15 unit.
- Baris 13: Metode `kurangiStok("B02", 12)` dipanggil untuk mengurangi stok barang dengan ID B02 (Meja) sebanyak 12 unit. Karena kuantitas fisik meja di gudang hanya tersisa 5 unit, kondisi logika internal menolak penarikan tersebut. Sistem menggagalkan transaksi mutasi ini, mencetak teks peringatan kegagalan di layar konsol, serta mencatat status pembatalan ke dalam daftar riwayat aktivitas gudang tanpa mengubah data stok asli.
- Baris 15: Eksekusi diakhiri dengan pemanggilan metode `cetakLaporan()`. Metode ini bertugas menampilkan rangkuman data mulai dari daftar kategori unik, visualisasi status sisa stok riil setiap barang pasca-mutasi, hingga urutan kronologis riwayat dari awal registrasi hingga penolakan transaksi terakhir.

## V. Kesimpulan

Berdasarkan hasil perancangan dan implementasi program simulasi Sistem Manajemen Gudang, dapat disimpulkan bahwa pemanfaatan Java Collections Framework (JCF) memberikan solusi manajemen data objek yang jauh lebih adaptif, dinamis, dan efisien dibandingkan penggunaan struktur array tradisional yang kaku. Integrasi tiga arsitektur utama JCF berhasil memenuhi kebutuhan fungsional operasional gudang secara optimal. Penggunaan `HashMap` sebagai basis data lokal terbukti sangat efektif karena karakteristik pencarian berbasis pasangan kunci-nilai (key-value) yang instan, sehingga mempercepat proses verifikasi data barang saat terjadi mutasi stok. Sementara itu, penerapan `HashSet` mampu menjaga integritas pengelompokan produk dengan mengeliminasi duplikasi nama kategori secara otomatis sejak barang didaftarkan. Terakhir, penggunaan `ArrayList` berhasil mengkompilasi seluruh jejak aktivitas logistik, baik transaksi yang sukses maupun yang gagal, secara terurut kronologis demi transparansi pengawasan data. Secara keseluruhan, kombinasi ketiga komponen ini bersama prinsip enkapsulasi pemrograman berorientasi objek murni mampu

menghasilkan sistem inventarisasi yang aman, terstruktur, dan andal di dalam memori program.

## **VI. Daftar Pustaka**

Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects (11th ed.). Pearson.

Schildt, H. (2018). Java: The Complete Reference (11th ed.). McGraw-Hill Education.

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek: Java Collections. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.